Memory Hierarchy

Topics:

- Cache
- Virtual memory

Memory - repetition

Collection of memory cells

- Memory cell:
  - Stores one memory word consisting of k bits (k = word size)
- Capacity
  - Number of cells * word size
- Control
  - Input for addresses
  - Input/output for data being read/written
  - Read/Write (RW) signal: determines whether memory is read or written
  - May be distributed over several memory modules: each module is activated by a chip select (CS) signal

Memory Hierarchy

Memory system development

Driven by contradicting aims:

- Speed (Typically: memory accesses distributed over several clock cycles)
- Size
- Cost

Memory Technology	Typical access time	Cost per Byte
S(tatic)**RAM**	Very low	Very high
D(ynamic)**RAM**	Low	High
Magnetic Disk	High	Low

How to build fast but still large/cheap memory systems?

Principle of Locality

Typical program behaviour:

Temporal locality (locality in time):

If an item is referenced, then it will tend to be referenced soon again.

Instructions: Loops
Data: Data just being calculated will be processed further

Spatial locality (locality in space):

If an item is referenced, items whose addresses are close by will tend be referenced soon.

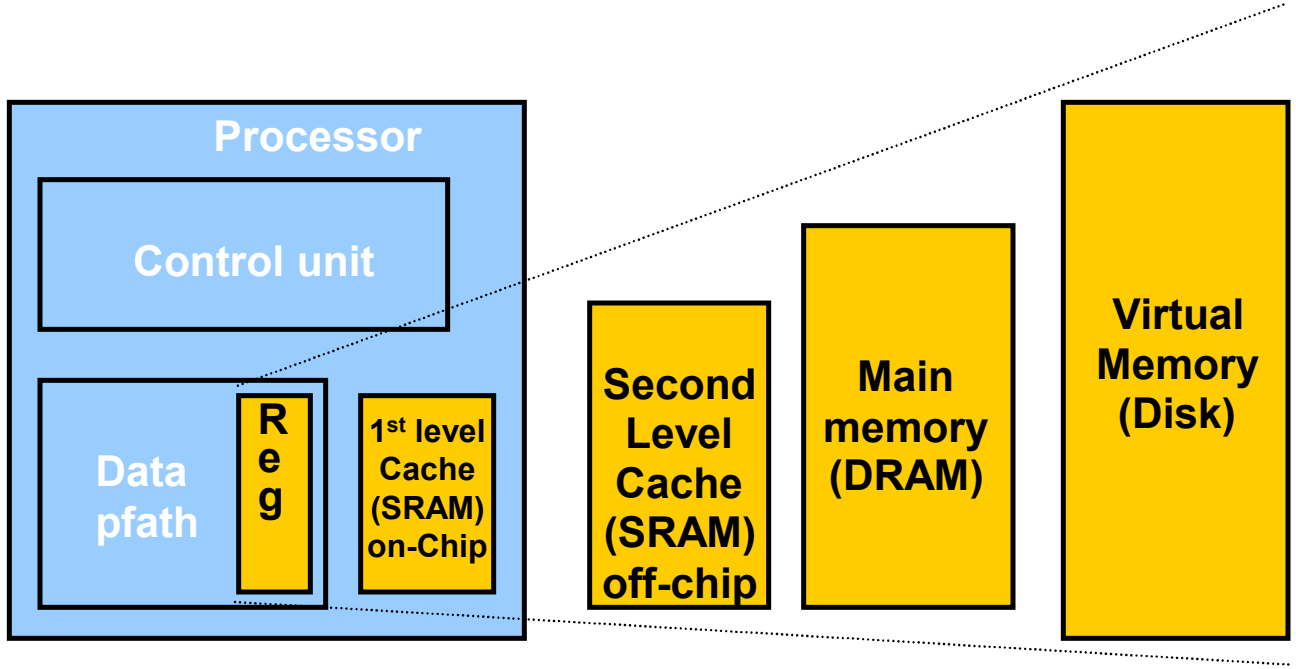
Instructions: PC most of the time is just incremented, jumps typically end in the nearer neighbourhood
Data: Loops iterating over arrays

⇒ During a given period of time only a small number of memory cells (typically being adjacent) are referenced.

Idea: copy their content into a small but very fast memory!

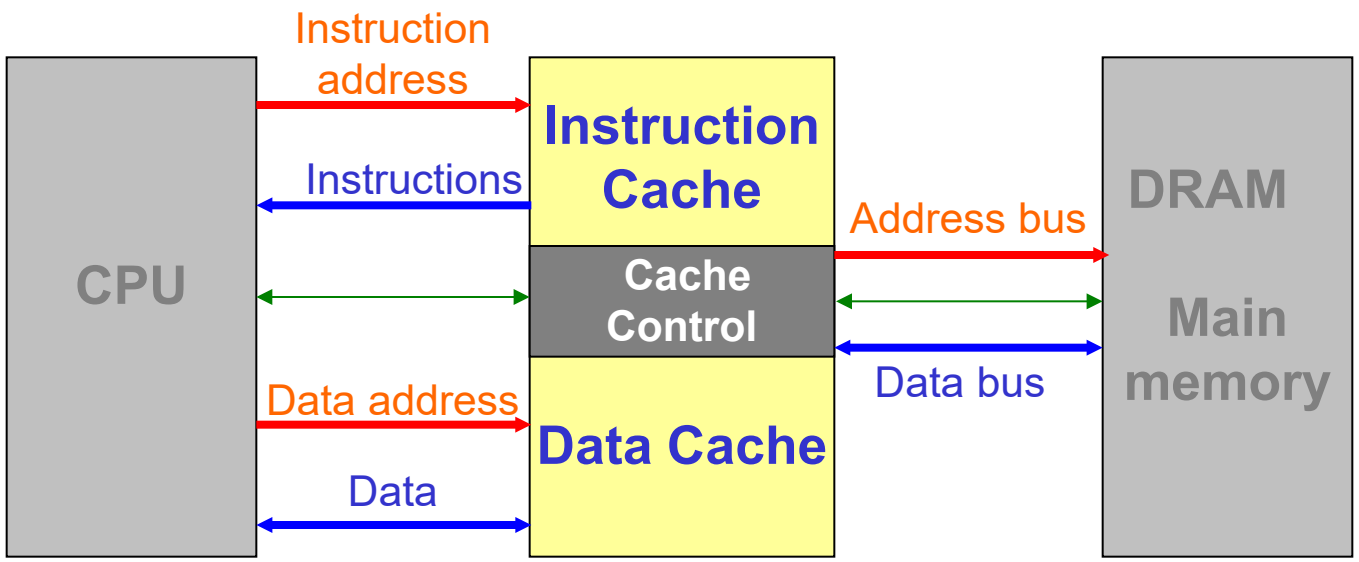
Memory Hierarchy

RAM: Random Access Memory
SRAM: Static RAM
DRAM: Dynamic RAM



Speed:	high	...	low
Size:	low	...	high
Cost per Bit:	high	...	low

Cache



Two separate caches: Instruction cache and Data cache

- Allows to read instructions and data item in parallel
- Allows to keep structure of caches more simple

Terminology

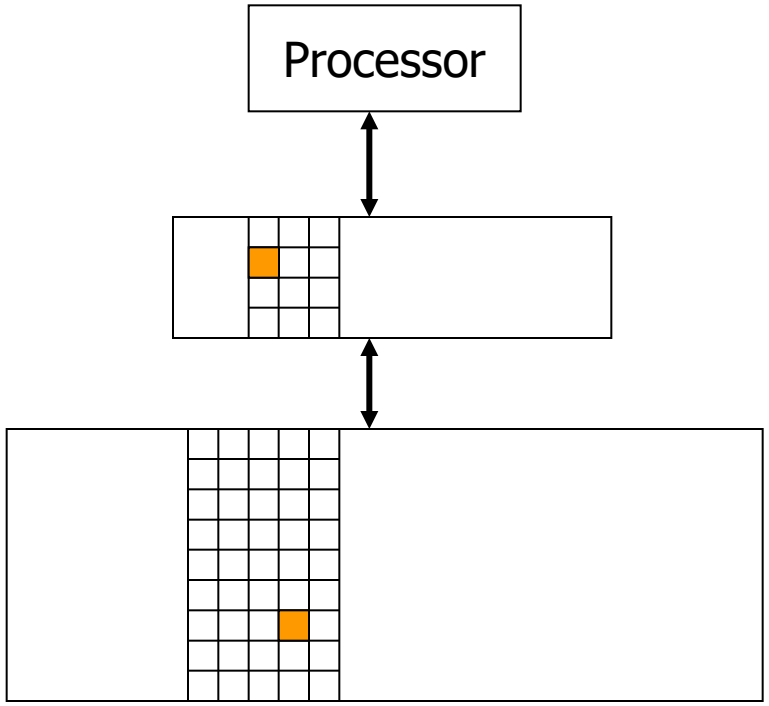
Memory hierarchy:

2 Levels: data is copied between adjacent levels

Upper level = *Cache* :
- Small
- Fast
- Expensive technology

Lower level = *Memory* :
- Large
- Slow
- Cheap technology

The minimum unit of information that can be either present or not present in one level is called a **block**.



Terminology

Data being requested by the processor is available in the upper level?

Yes: *hit*

No: *miss*

Fraction of memory accesses found in the upper level: *hit ratio*

Fraction of memory accesses not found: *miss ratio*

Time necessary for accessing the upper level: *hit time*

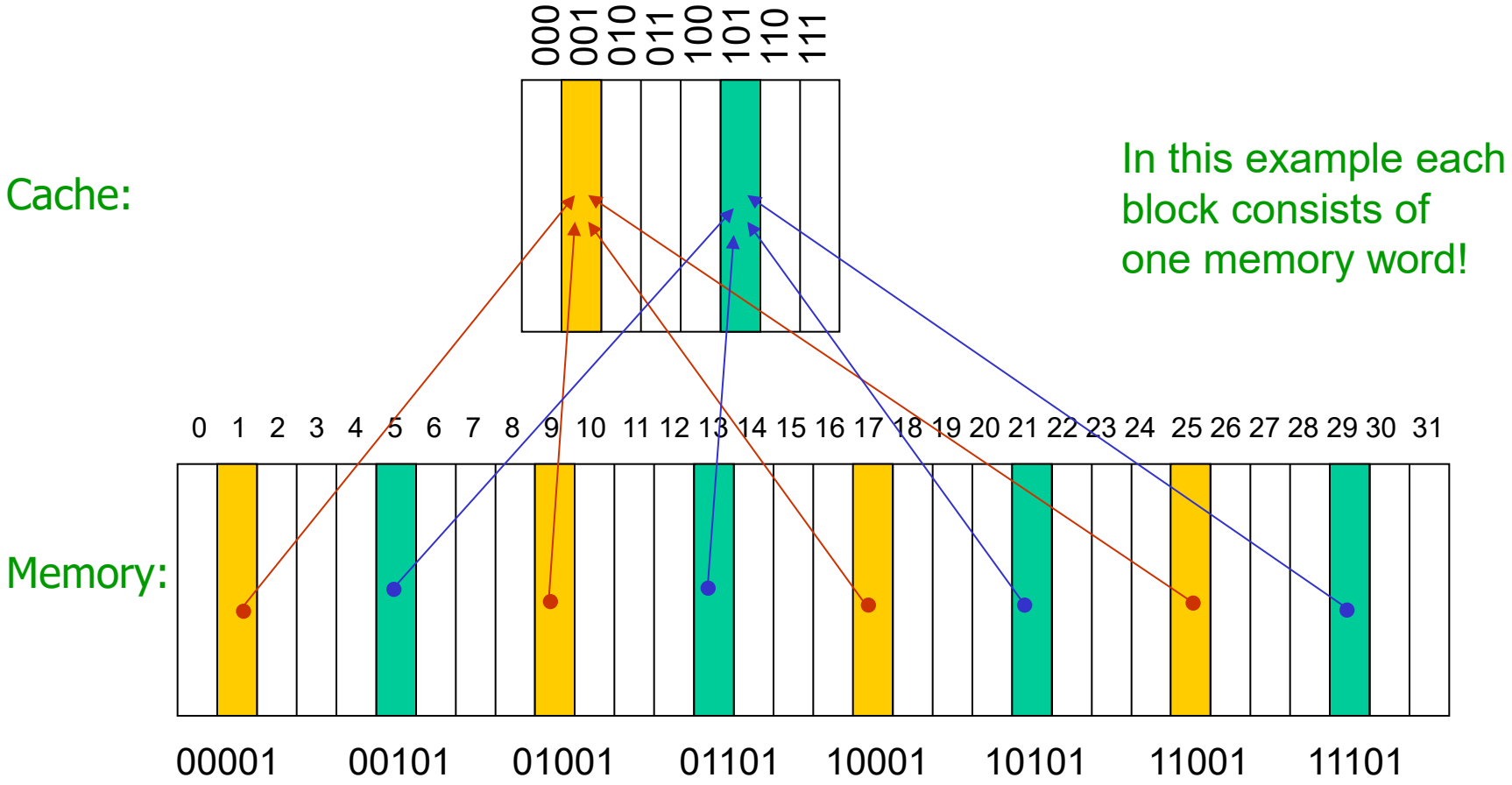
(Including time for determining whether it is hit or a miss)

Additional time being required for accessing the lower level in case of a miss: *miss penalty*

Direct Mapped Cache

For each memory cell there exists exactly one location in the cache where it can be stored, e.g. the memory address of each block can be mapped only onto one cache block address (= Index).

Index = (Memory address of block) modulo (Number of blocks in the cache)



Addressing Data in the Cache

Each cache location can contain the contents of several different memory locations. How do we know which one?

Tag: For each block, the part of the upper portion of the address being the same for all memory cells in this block is stored.

If one block contains multiple words, how do we find the desired one?

Block offset: Number of a word inside of a block

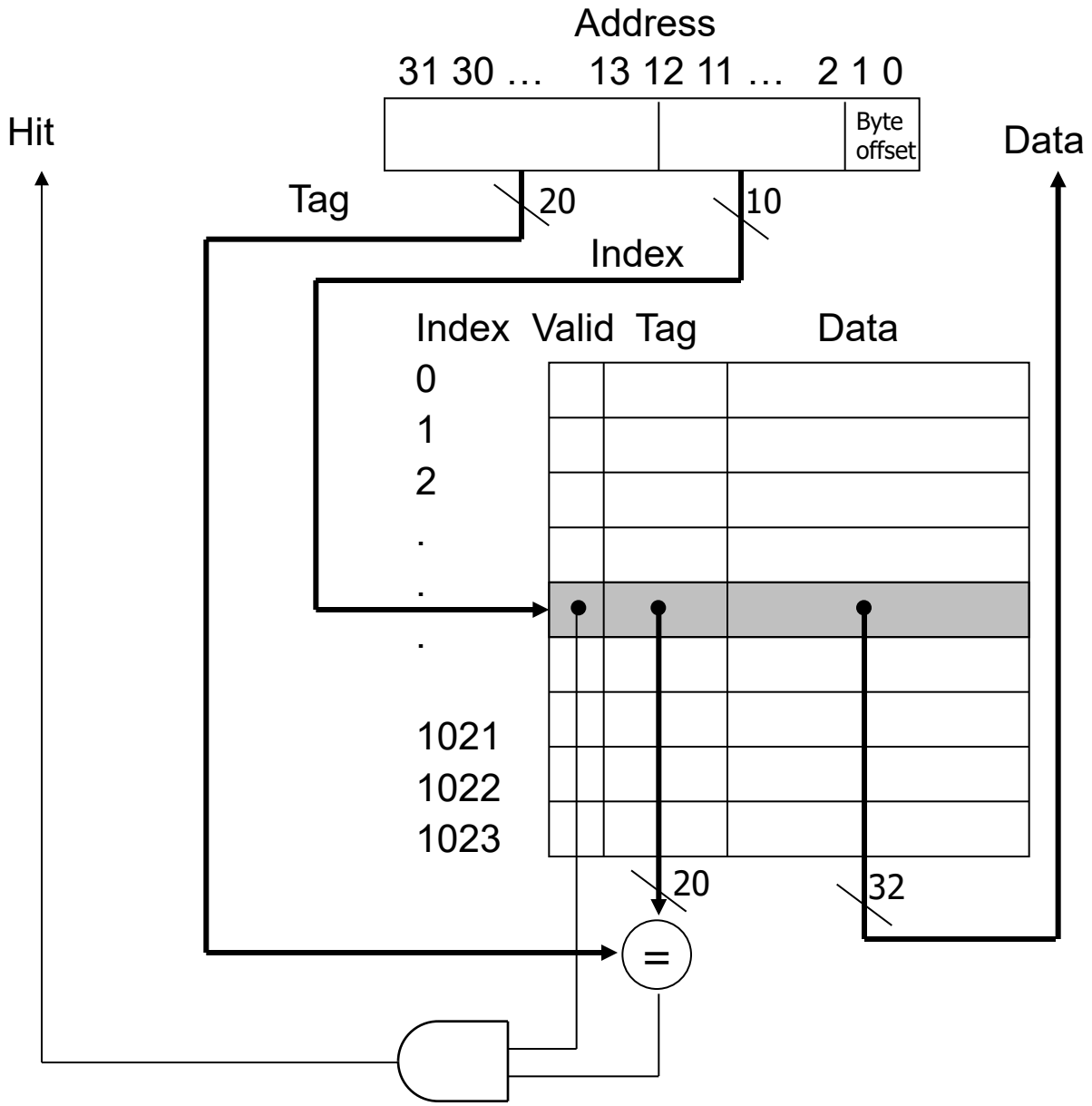
Tag	Index	Block offset

Parts of the address used for addressing the cache

Cache may contain invalid data, e.g., after system start when the cache is empty:

Valid bit: indicates whether content of block is valid

Addressing a Cache



Byte offset:
Word size = 32 bit
⇒ 2 LSB not used

Block offset:
one word/block
⇒ no block offset

Reading from the Cache

Read hit: an instruction is executed as described in sections 5 and 6.

Read miss: cache control unit notifies CPU that the item it just has read has been invalid ⇒ CPU has to be stalled!

Actions to perform in case of a read miss:

Example: read miss on instruction cache; data cache read misses are handled in a similar way.

1. Send original PC (current PC – 4) to memory
2. Instruct memory to perform a read and wait for the memory to complete its access.
3. Store data in cache, store upper portion of address in tag field and set valid bit to one.
4. Restart fetching the instruction

Writing to the Cache

Write hit:

Question: How to deal with / avoid inconsistencies between memory and cache?

1. Write through method:

The information is written to both the block in the cache *and* to the block in the next level memory

+ Easy to implement
- No speedup for writes
  Solution: Additional write buffer between cache and memory

2. Write back method

The information is written only to the block in the cache. The block is marked as modified (= dirty) using an additional *dirty bit* for each block.

The data is written back to memory later when the block is replaced in the cache.

Write back versus write trough

Advantages of write back:

- Enables writing to the cache without stalling the processor in case of a hit
- Reduces the total number of writes to main memory ⇒ reduces the working load on the system bus.

Drawbacks of write back:

- In cases where multiple components of the computer are allowed to write to memory more sophisticated techniques are necessary for solving the cache coherency problem.

  Examples:
  - Multi-core architectures with a shared memory
  - IO devices being allowed to access memory (direct memory access, DMA)

Cache coherency problem: A cache block may contain different data than the corresponding cells in memory

Writing to the Cache

Write miss:

1. Write allocate method:

The corresponding block is allocated first. Then the information is written as in the case of a write hit.

Typically used in combination with *write back*

2. Write no-allocate method:

The information is written only to the memory. The cache is not affected.

Typically used in combination with *write through*

Taking advantage of spatial locality

Spatial locality: If an item is referenced, items whose addresses are close by will tend to be referenced soon.

Idea: in case of a cache miss load multiple words from memory (and not only the one being referenced)
$\Rightarrow$ multiple words for each cache block

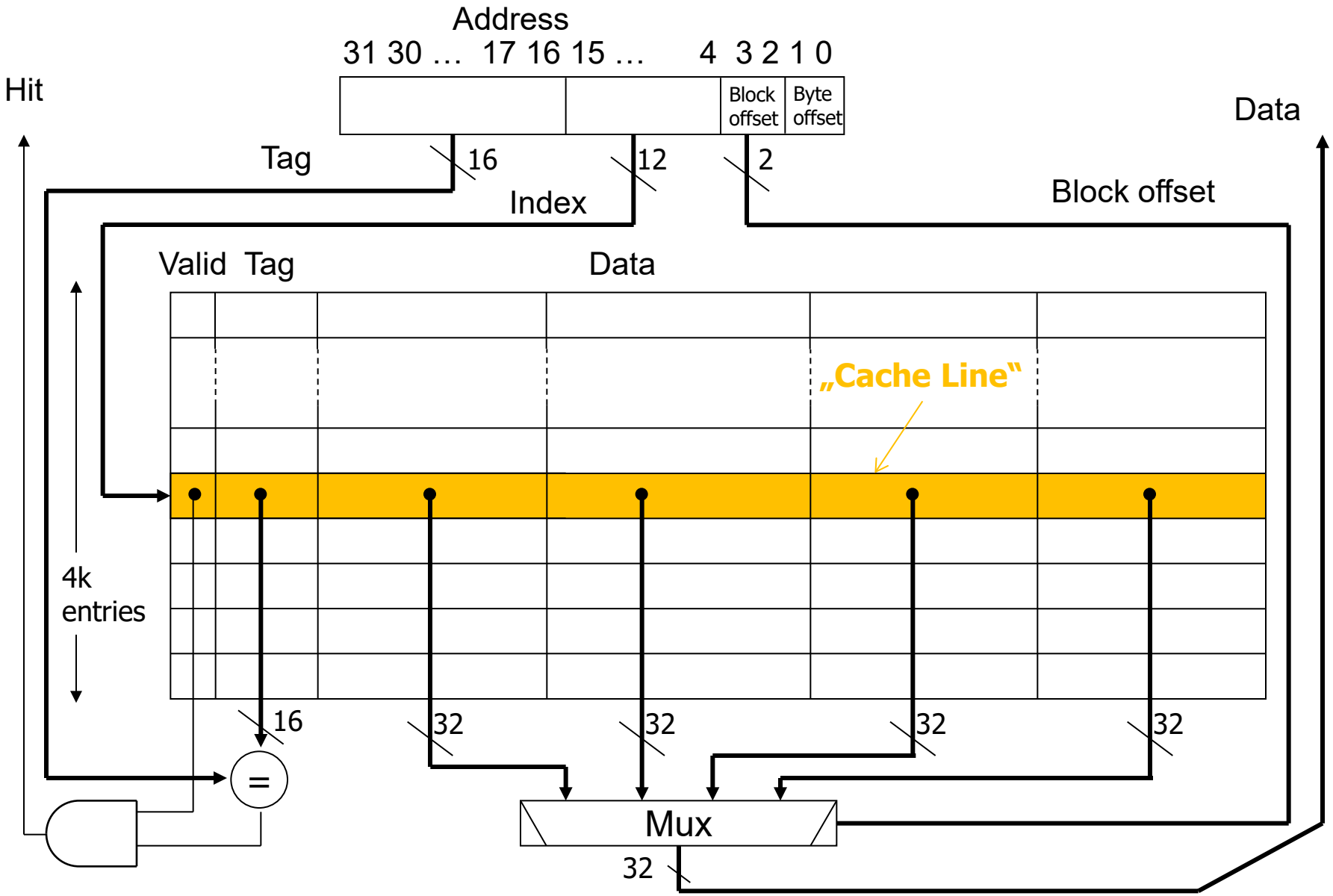
In case of a miss multiple words are read from/written to memory!

Example:

Consider a 64 Kbyte cache with **four** 32 Bit words for each block

- 4 Byte words, 4-word blocks $\Rightarrow$ 16 Byte per block $\Rightarrow$ 4K (= 4096 = 2^{12}) blocks
- 12 Bit block address (= Index)
- 2 Bit block offset
- 32 - 12 - 2 - 2 = 16 Bit tag

64 KB cache using four-word blocks



Designing the memory system to support caches

To take advantage of grouping multiple words into one block, the memory must support multi-word memory accesses in order to increase the *memory bandwidth* (= number of Bytes being transferred per clock cycle).

Example:

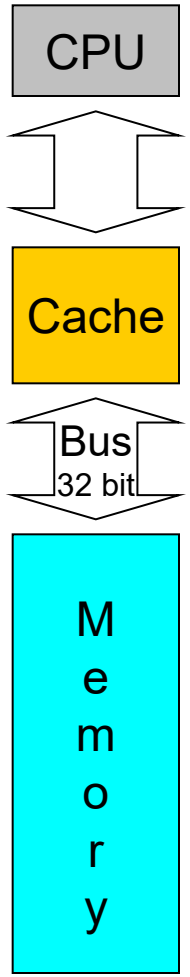
One block contains 4 words

Assume that reading data from memory in case of a *cache miss* takes the following times:

- 1 clock cycle for sending the address to memory
- 15 clock cycles for reading data from memory (DRAM technology)
- 1 clock cycle for sending data from memory to the cache

How much time does it take to reload one block?

One word wide memory organisation



1 word wide organisation

1. Cache sends address *a* to memory
2. Memory reads and sends back the words stored at *a*, *a* + 4, *a* + 8, *a* + 12 one after the other

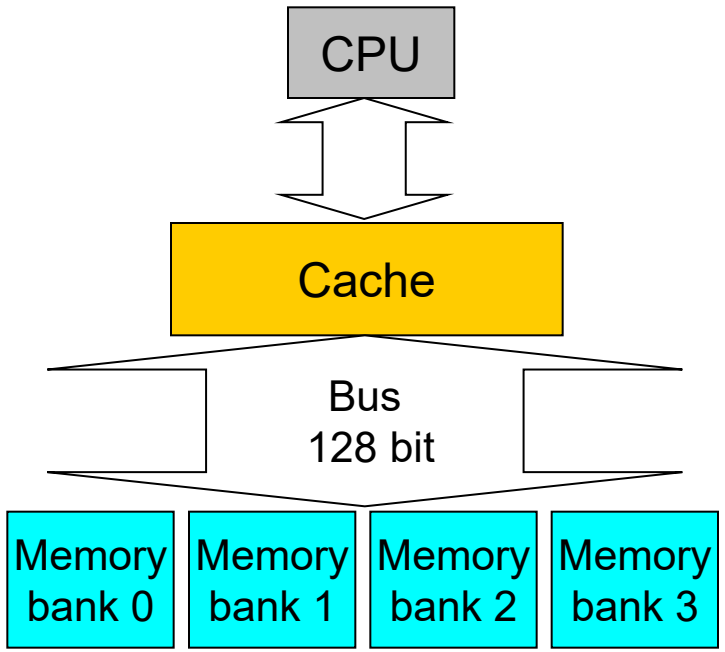
Miss penalty:

$1 + 4 \cdot 15 + 4 \cdot 1 = 65$ clock cycles

4 words = 16 bytes

Band width: 16 / 65 = 0.25

Four word wide memory organisation



Wide memory organisation

Memory data bus is extended to 128 bit in order to enable accessing four words in parallel:

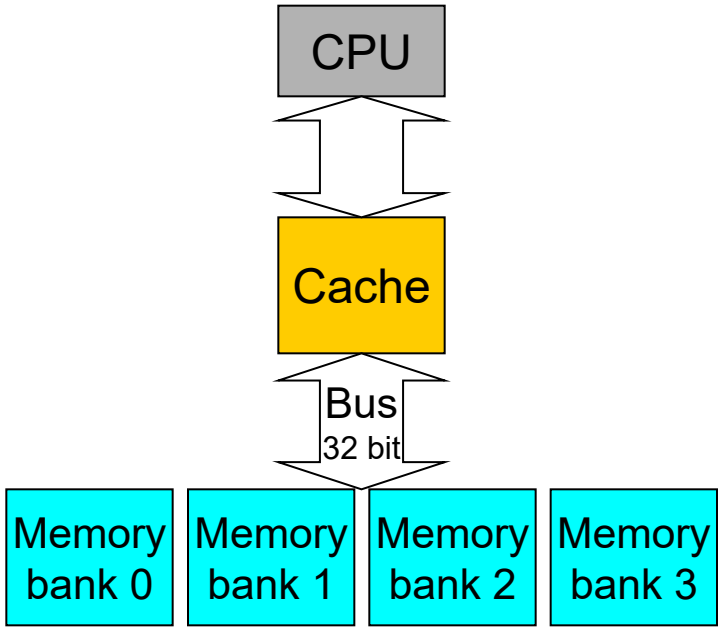
1. Cache sends address *a* to memory
2. Memory reads and sends back the words stored at *a*, *a* + 4, *a* + 8, *a* + 12 in parallel

Miss penalty: 1 + 15 + 1 = 17 clock cycles

4 words = 16 bytes

Band width: 16 / 17 = 0.94

Interleaved memory organisation



Interleaved memory organisation

Memory is organized in banks (each being one word wide) that can be addressed and accessed in parallel:

1. Cache sends address *a* to memory
2. Memory reads the words stored at *a*, *a* + 4, *a* + 8, *a* + 12 in parallel but sends them to the cache sequentially

Miss penalty:

$1 + 15 + 4 \cdot 1 = 20$ clock cycles

4 words = 16 bytes

Band width: 16 / 20 = 0,8

Where can a block be placed in the cache?

Direct mapped (one-way associative) cache:

- One location for each block
- Number of block is calculated from index
- Tag of the selected block and tag of the address are compared

  ⇒ only one comparator necessary
- Involves the possibility of *conflicts* :

  Example: cache with 16 one-word blocks
  word 1, 17, 33, … go to the same block!
  ⇒ may discard each other leading to *collision misses* !

Fully associative cache:

- Each block can be placed anywhere in the cache
- When accessing the cache, the tags of all blocks are compared in parallel against the tag of the address

  ⇒ no conflicts

  But: one comparator for each cache block necessary ⇒ expensive!

  ⇒ Only used for small caches!

Where can a block be placed in the cache?

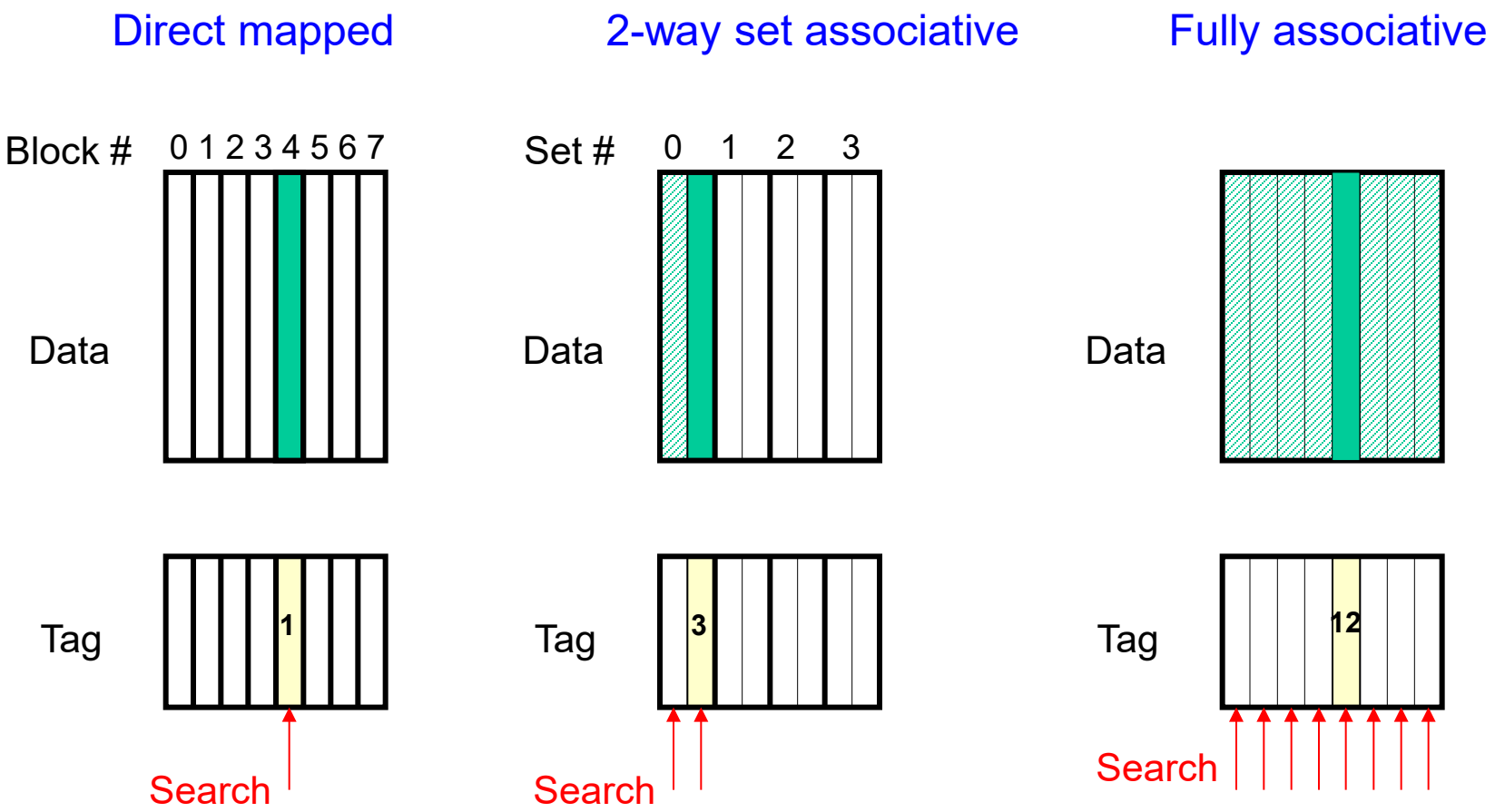
Set associative cache:

- For each memory block there exists one set of cache blocks where the memory block can be placed
- For a *n* - way set associative cache each set consists of *n* cache blocks
- The index field denotes the number of the set where a block can be placed:

  Index = (Block address) modulo (number of sets)
- A memory block can be placed in any block of the corresponding set!
  ⇒ *n* comparators necessary

Where can a block be placed in the cache?

Example: block with memory address 12 (`1100`) is stored in a cache using 8 blocks



12 mod 8 = 4

12 mod 4 = 0

Block placement strategies

Example: caches using 8 blocks

One-way set associative (direct mapped)

Block	Tag	Data
0		
1		
2		
3		
4		
5		
6		
7		

2-way set associative

Set	Tag	Data	Tag	Data
0				
1				
2				
3				

4-way set associative

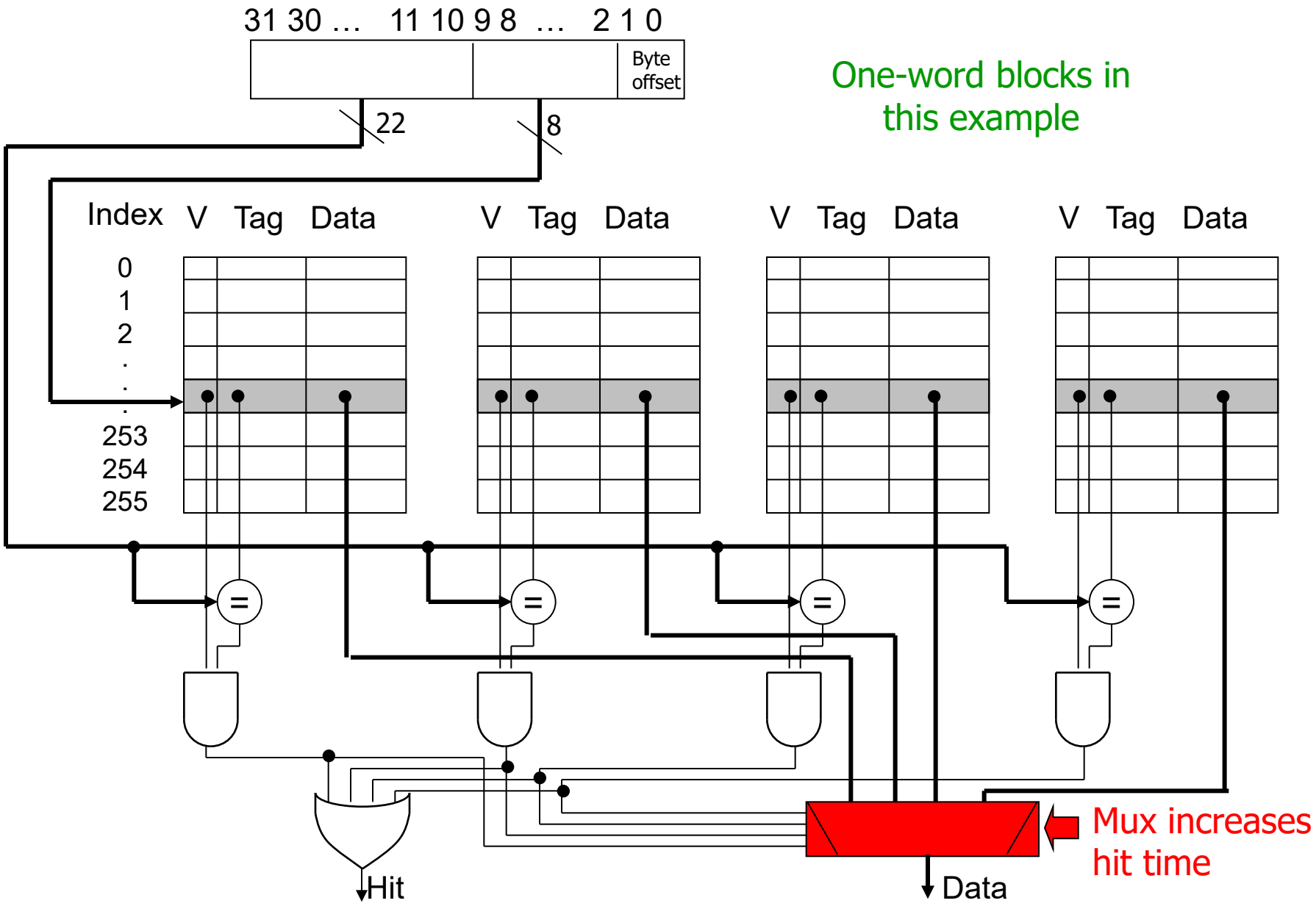
Set	Tag	Data	Tag	Data	Tag	Data	Tag	Data
0								
1								

8-way set associative (fully associative)

Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data	Tag	Data

Number of blocks = number of sets · associativity

Implementation of a 4-way set associative cache



Block replacement strategies

If a block may be placed at several locations, which location should we select, i.e., which of the blocks in the cache should be replaced?

1. Random strategy: Randomly select the block that will be replaced.

   + Easy to implement

2. Least recently used (LRU) strategy:

   Replace the block of a set that has been unused for the longest time.

   + Reduces number of conflict misses

   HW implementation: one counter c_i for each block b_i

   when accessing block b_i

   1. Increment counters of all other blocks of the set
   2. Set c_i to 0

   The block with the largest c_i will be replaced

   + Still easy to implement for 2- and 4-way set associative caches

   - Quite expensive for fully associative caches

Main memory

- Modern computer have a very large address range
- Using *n* address bits we can address 2^n memory cells

Number of address bits	Number of addresses	Cost for main memory using the whole address range (5 €/1GB)
16	65536	--
32	$4{,}3\times10^9$	20 €
64	$1{,}8\times10^{19}$	8.5×10^{10} €

⇒ We cannot afford to provide computers with memories being as large as the address range!

Multi tasking systems

Typical workstation scenario:

Many programs run at once (many operating system programs, several user programs, maybe started by different users)

Consequences:

- A program cannot use the whole memory for itself
- Programs have to be protected against each other: A program must not access portions of memory that have been assigned to other programs.

Problem: Compilers/linkers do not know

1. how much physical memory is available and
2. where in memory a program will be stored!

Multi tasking systems

Possible solutions:

- Loader replaces absolute addresses (e.g. in jumps) when loading a program
- User programs may only use addresses relative to PC or dedicated address base registers

Drawbacks:

Fragmentation:

Unused memory may be split into many small portions making it impossible to allocate large contiguous blocks in memory

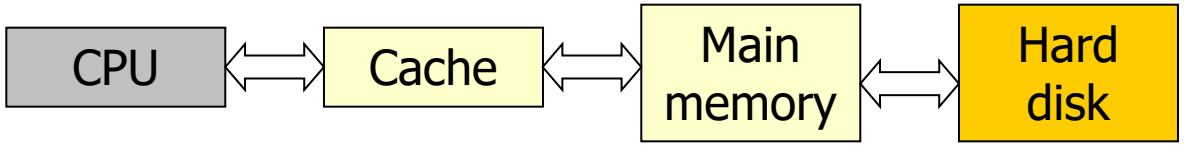
Low utilization:

The total memory required for all programs must not be larger than the amount of memory being available on the machine.

But: only a fraction of memory may be used at any point in time!

Virtual Memory

Extend memory hierarchy by one additional level: hard disk



- Data that cannot be stored in main memory because of capacity reasons are moved to the hard disk

  ⇒ Capacity of the whole virtual memory system may be much larger than main memory!

- From the user point of view all programs and all data are stored in the main memory
- Moving portions of data from/to disk is performed by the operating system
- Requires assistance by the hardware

Virtual Memory

Each program is allowed to use the whole address range (independently on how much memory is actually present, and which other programs are running at the same time)!

- **Process**: unique execution of a program, process is defined by both its code and its data, two copies of a single program executing on their own data constitute two distinct processes
- Addresses used by a process are translated into real hardware addresses
- Hardware addresses may point to main memory or to disk
- The address spaces are managed by the operating system:
  - Which data should be stored in main memory?
  - Where in memory should data be stored?
  - Which data should be swapped to the disk when we need to allocate additional space for data?
- Translating the addresses is done by operating system and additional hardware.
- Basic concept: *Paging*

Some basic concepts of virtual memory are very similar to the cache concept. However, because of historical reasons a different terminology is used!

Paging

Basic Idea:

- Memory as it is seen by a program = virtual memory; it is subdivided into pages of fixed size (e.g. 4kB)

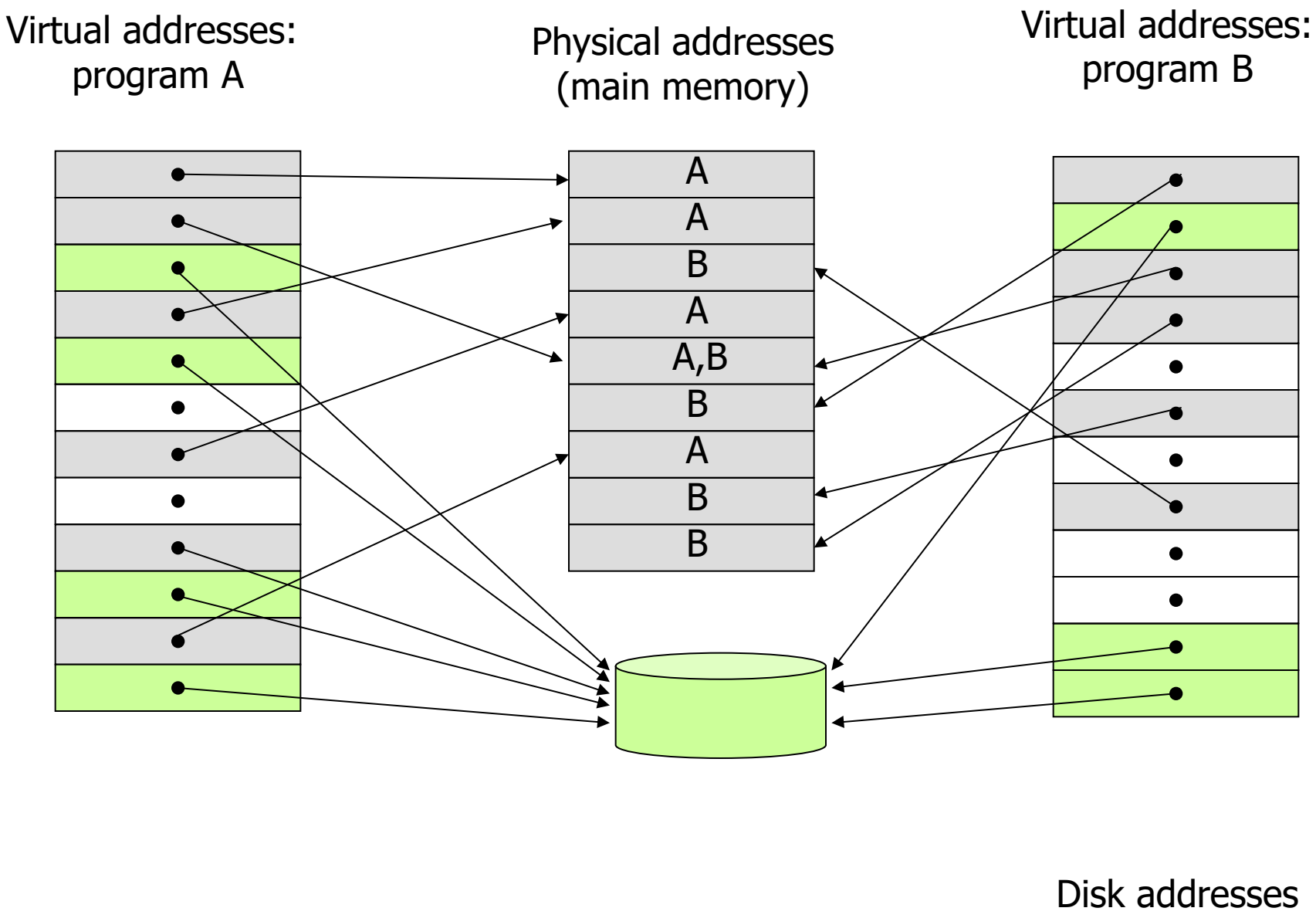
- Main memory (DRAM) = physical memory; the parts of the main memory where pages can be stored are subdivided into page frames.

- CPU uses virtual addresses for addressing data. These addresses are translated by the operating system and dedicated hardware into physical addresses.

- One physical address may be assigned to several virtual addresses to allow several programs to share data and parts of the program (e.g. library routines). Access to shared memory is supervised by the operating system.

A "block" of a cache corresponds to a "page" of virtual memory,
a "cache miss" corresponds to a "page fault".

Virtual and physical address



Virtual and physical address

Virtual memory simplifies loading of a program for execution:

1. A program may be stored anywhere in the physical memory
2. The operating system does not have to allocate a contiguous block of memory for a program.

⇒ Relocation of a program

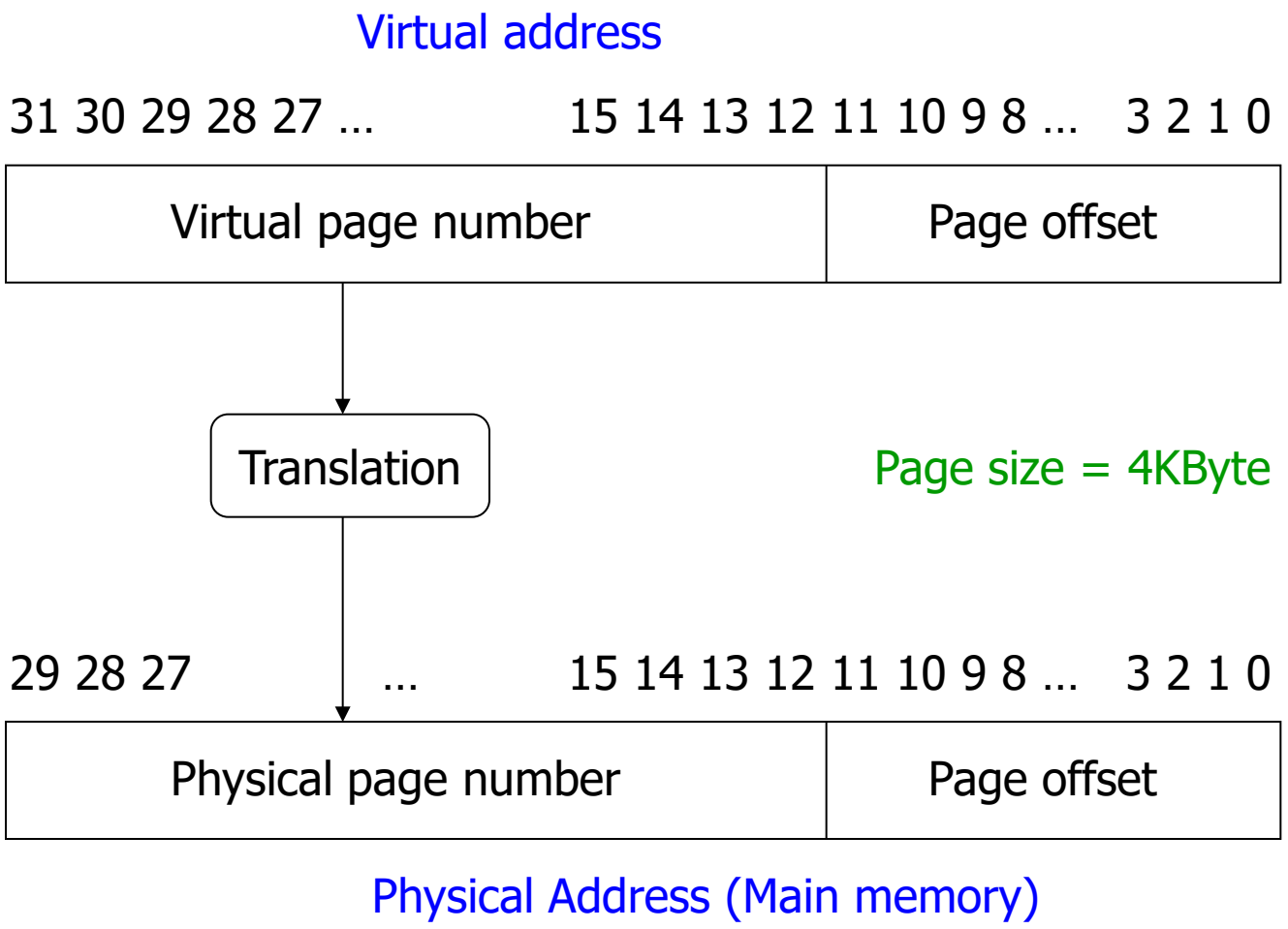
A virtual address consists of two parts:

1. Virtual page number: selects one page
2. Page offset: selects one memory cell within a page.

A physical page number points
either to a page frame in main memory
or to a page on the disk.

Virtual and physical address

Example: 32-bit address, 1 GB physical memory



Page faults

Page fault: If the processor wants to access data being not in main memory, the corresponding page has to be loaded from disk.

Very expensive - loading a page from disk requires millions of clock cycles!

⇒ Avoid reading/writing to disk whenever possible!

Page faults can be managed by software: Slower than hardware solution, but time for accessing disk still dominates.

⇒ Facilitates usage of sophisticated techniques for controlling loading/saving of pages:

1. Fully associative placement of pages in main memory: allows to apply advanced strategies for replacing pages in main memory

   ⇒ Reduces number of conflicts, e.g., number of pages that have to be reloaded from disk

2. Write back method for saving pages that have been modified; for each page one valid-bit (page in memory) and one dirty-bit (page is modified)

   ⇒ Reduces number of pages being written to disk

Page table

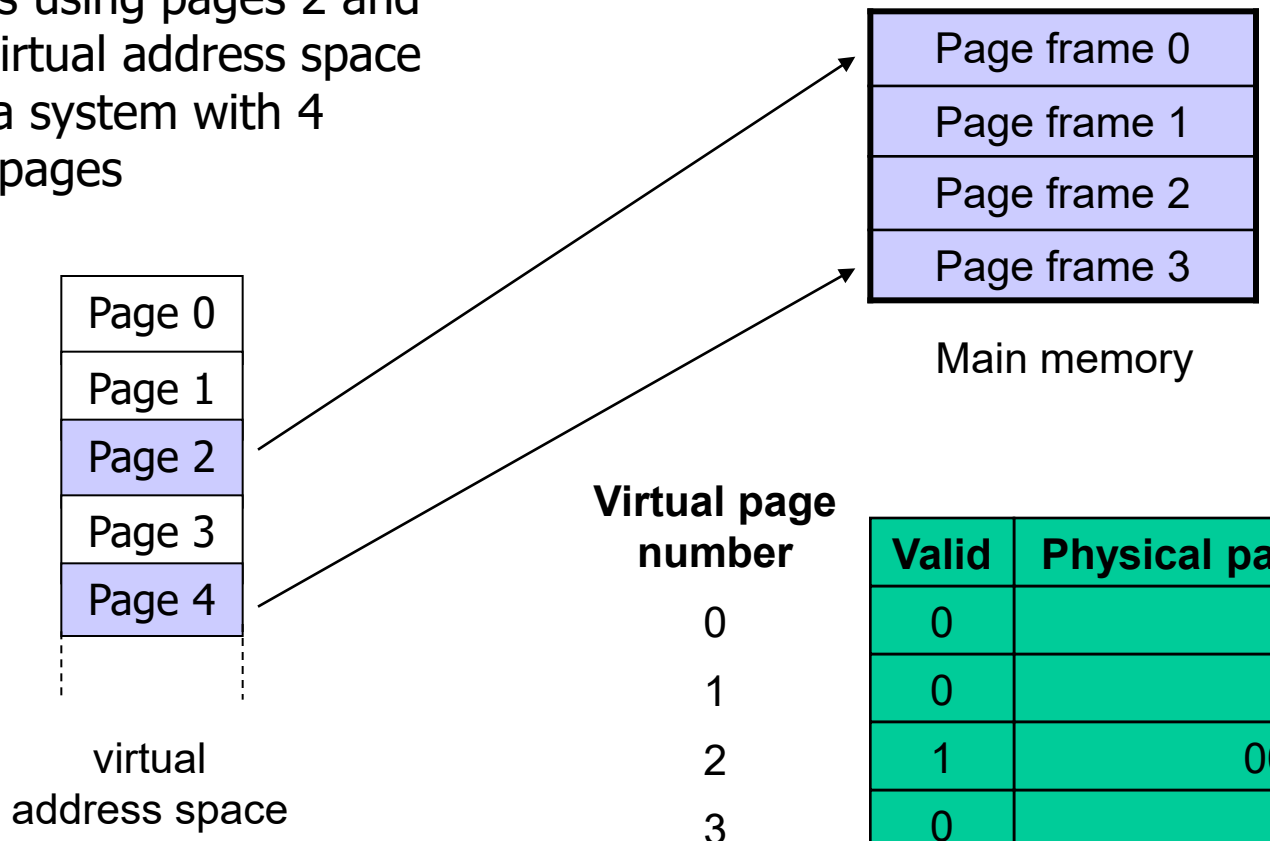
For translating virtual page numbers into physical page numbers a *page table* is used:

- Contains for each virtual page number the corresponding physical page number and the valid bit
- Page table is stored in main memory
- Each process has a page table of its own
  $\Rightarrow$ allows to provide each process with a full address space of 2^{32} addresses of its own
- Processors that support virtual memory have a *page table register* that contains the start address of the page table of the program being active

Paging

Example:

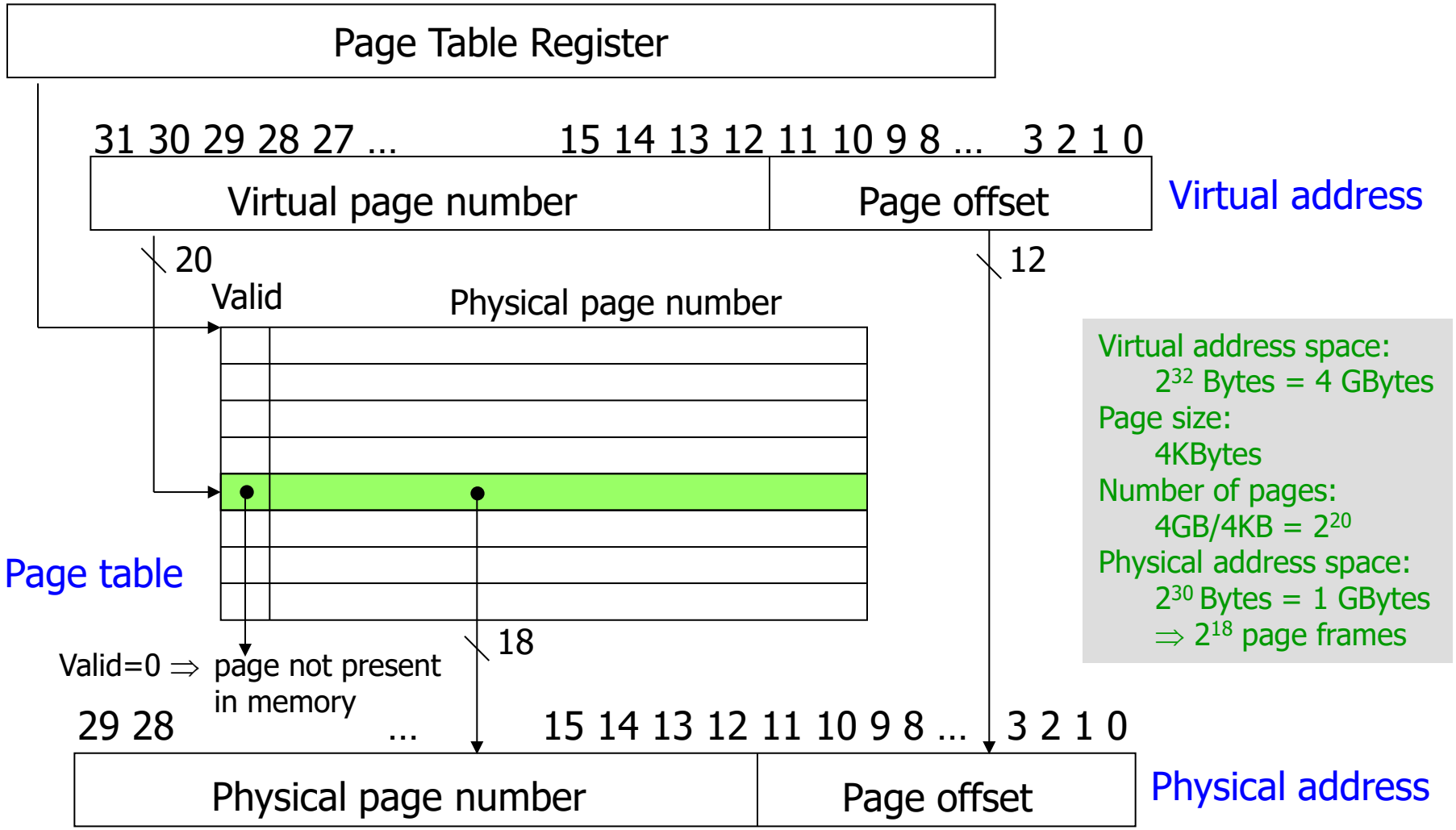
A process using pages 2 and 4 of its virtual address space runs on a system with 4 physical pages



Virtual page number	Valid	Physical page number
0	0	
1	0	
2	1	00
3	0	
4	1	11

Page table (in main memory)

Determining the Physical Address



Virtual address space:
2^{32} Bytes = 4 GBytes
Page size:
4KBytes
Number of pages:
4GB/4KB = 2^{20}
Physical address space:
2^{30} Bytes = 1 GBytes
⇒ 2^{18} page frames

Hardware to support virtual memory:
Memory management unit (MMU)

Process switch

Process: program being executed on a CPU
Multi-tasking systems:

Operating system allows to share CPU and memory with several processes by activating and deactivating them one after another
⇒ process switch or context switch

A process is activated by loading its state into the processor.

The state of a process contains:

- PC
- Register contents
- Page table (reload page table register)

Who modifies the page table?

The page table contains information about
1. which pages of the virtual address space are used by a process and
2. where the corresponding physical pages are located in the main memory
⇒ Denotes which physical pages belong to the process

Only the operation system is allowed to modify page tables!

Modifying the page table

Starting a program:

The operating system allocates physical pages in memory (and disk, if necessary) and builds an initial page table

Allocating memory:

- If a process wants to allocate additional memory for storing data during runtime, it calls an operating system routine. The operating system then allocates additional pages and updates the page table.

- If the stack has to be enlarged, additional pages are allocated.

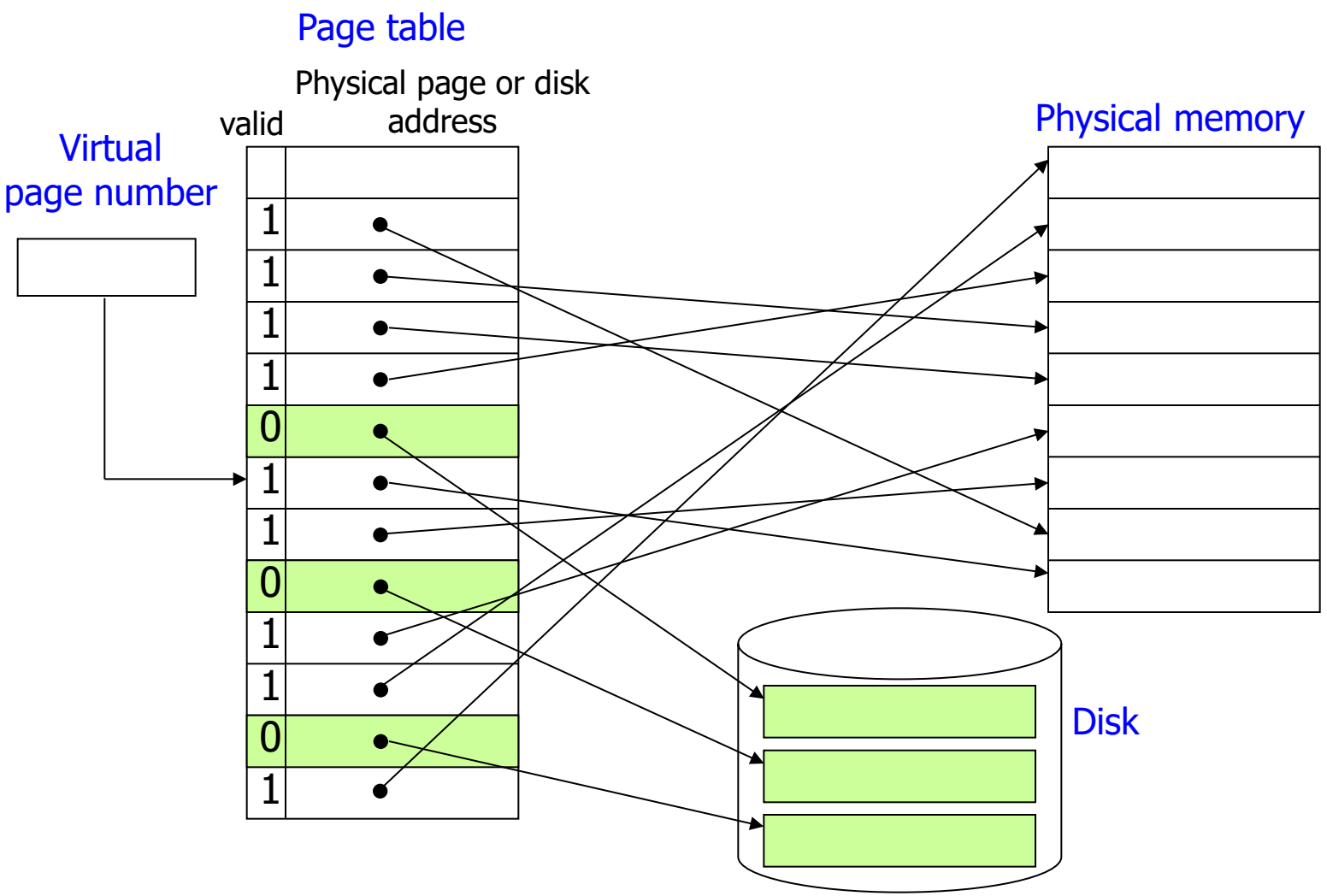
If the operating system has to remove pages of a non-active process from main memory, it has to update its page table.

⇒ The operating system needs information about which processes use which physical pages!

The operating system creates an additional table that denotes for each physical page which virtual page of which process is mapped to it.

Virtual memory using a single page table

Where to store addresses of pages on disk?



Disk addresses

During loading a page from disk the corresponding page table entry is overwritten by the new memory address.

Single-entry page table:

The old disk address gets lost. When the page will be removed from memory later, it must be written back to disk even if it has not been modified.

⇒ To prevent unmodified pages from being written back to disk we have to store both, the physical memory address and the disk address!

Implementation alternatives:

1. Extend page table entries: Additional data fields for storing disk address

2. Two page tables:
   One for physical memory page addresses and one for disk page addresses

   If both tables do not contain a valid page address:
   ⇒ Virtual page has not been allocated yet for this process!

Page Fault

If the valid bit of a page being accessed is 0, a page fault occurs, e.g., the page table does not contain a valid physical memory address for the virtual page address being requested.

CPU causes an exception, and operating system performs reloading the page.

```
Check whether page being requested is located in physical memory (valid=1)
if (page fault)
then {
        Check whether there are unused physical page frames
        if (no page frame empty)
        then {
            write-back a page from physical memory;
            update the page table of the corresponding process
        }
        place requested page in empty page frame
        update page table of active process;
}
access page in memory;
```

Replacement strategies

LRU strategy (Least Recently Used)

Replace the page that has been unused for the longest time.

Completely accurate implementation:

Complex data structure necessary – would have to to be updated on every memory access

⇒ too expensive!

Simple implementation:

Use one Reference bit for each page:

- is set by MMU whenever the page is accessed
- Operating system periodically records and clears the reference bits so it can determine which pages are touched during a particular time period
- replaces one page being among the least-recently referenced.

Translation Lookaside Buffer (TLB)

Problem:

Each access of a program to its virtual memory requires two accesses to the physical memory:

1. One access to page table for determining the physical address
2. One access to data/instruction in main memory

May increase the time spent for memory accesses!

Solution:

use an additional cache to store the physical page addresses of frequently used virtual addresses:

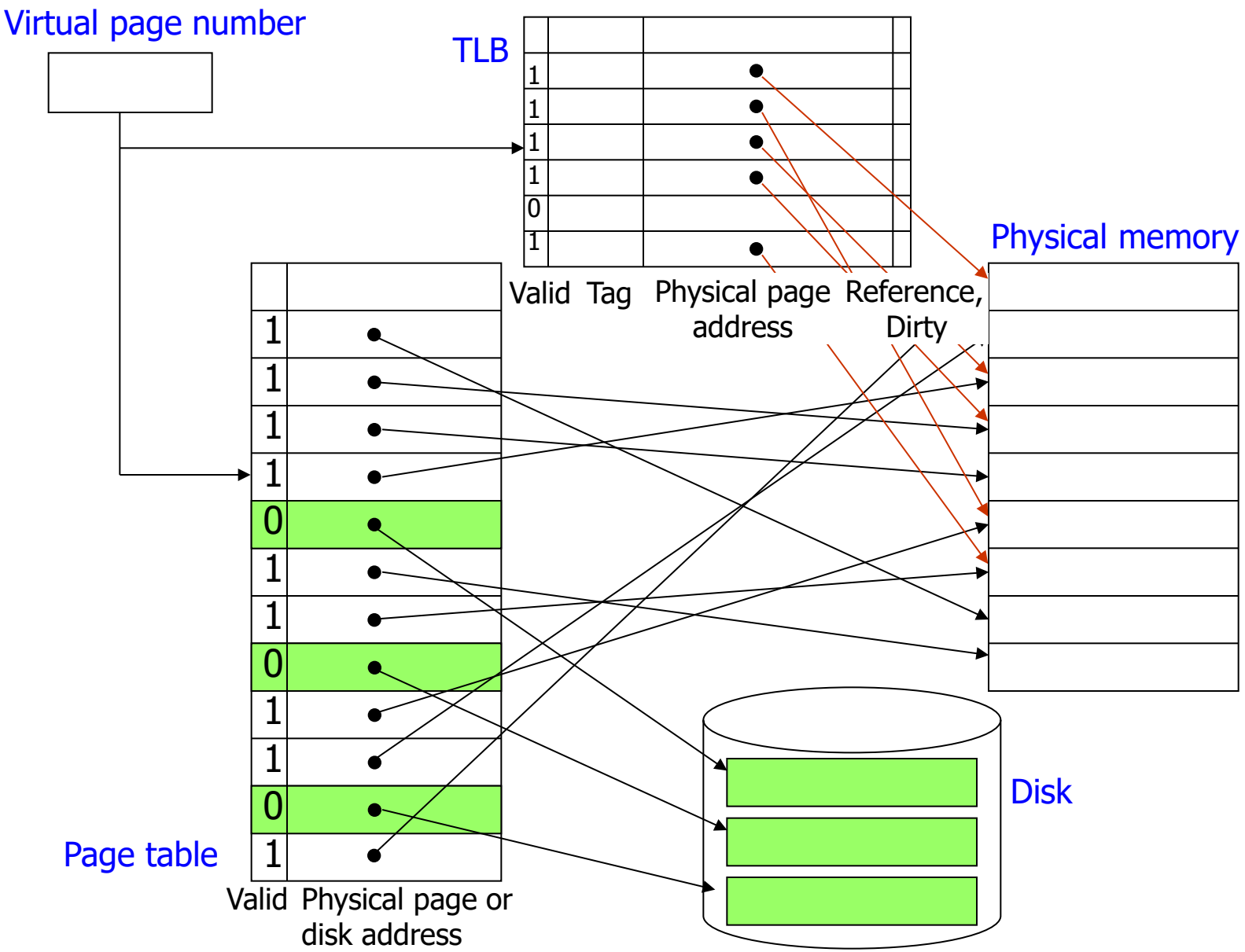
Translation look-aside buffer (TLB)

May be organized as direct mapped, set or fully associative cache

Block: physical page entries (including status bits and reference bit)
Tag: MSBs of virtual pages address
Valid bit: Indicates whether TLB entry <u>and</u> the corresponding page is valid

Write-back strategy for dirty and reference bits

Translation lookaside Buffer (TLB)



Translating a virtual address

Virtual address: lookup physical page number in TLB

TLB hit:
Set reference bit
Set dirty bit if writing

TLB miss: lookup physical page number in page table

Page fault:
CPU causes exception
Operating system reloads page

hit: update TLB entry

A TLB miss can be managed by the hardware
- either autonomously
- or by using an exception.

In the latter case the TLB is updated by the operating system

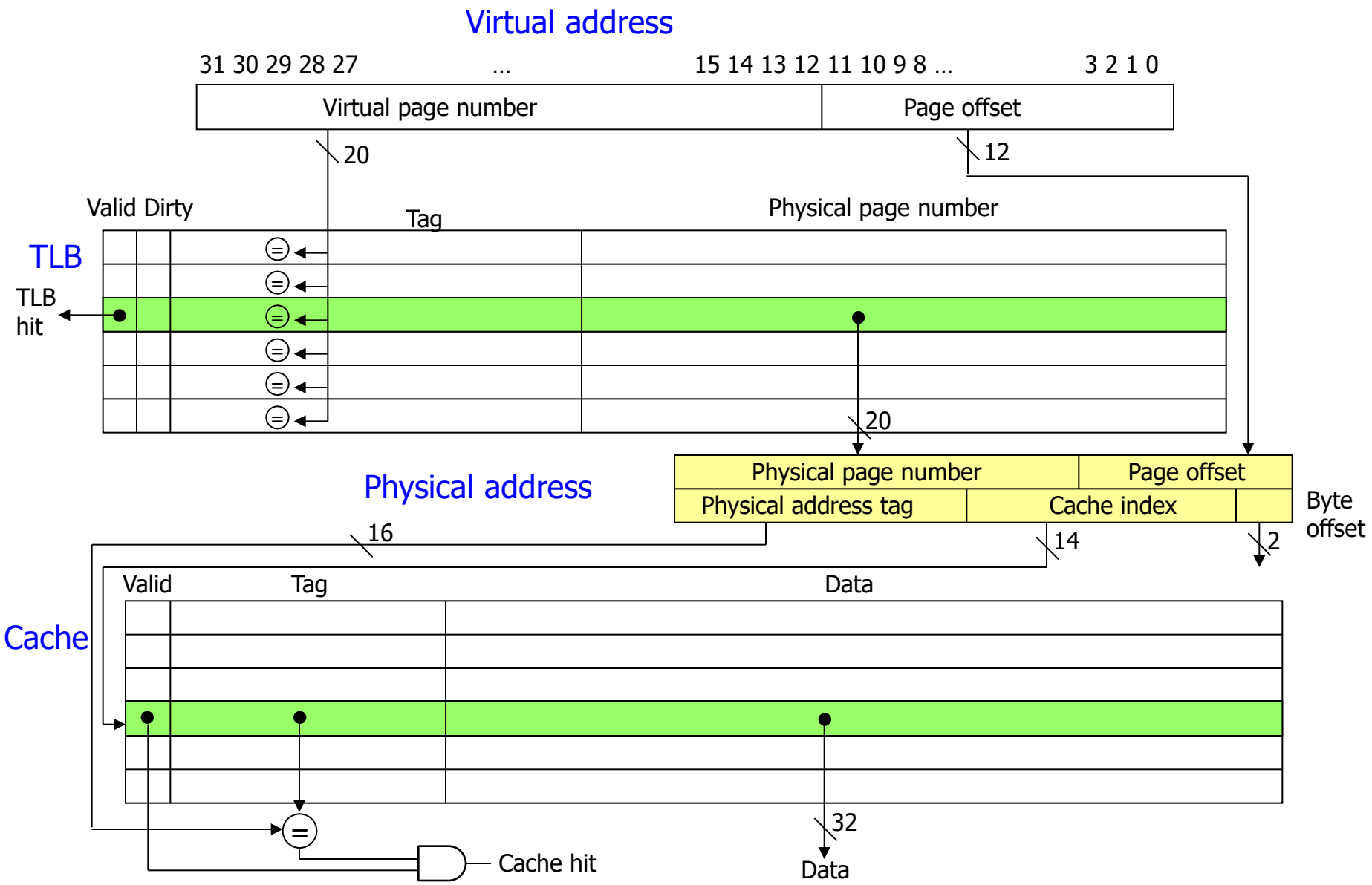
Integrating virtual memory, caches and TLB

Example: MIPS R2000 TLB

- 32-bit address space, 4KByte page size
  $\Rightarrow$ 20 bits for virtual page address
- Fully associative TLB with 64 entries used for instructions and data
- The physical address is used to access a single direct mapped cache used for instructions and data.

$\Rightarrow$ In case of a hit, accessing virtual memory requires one access to the TLB and one access to the cache

Example: MIPS R2000 TLB



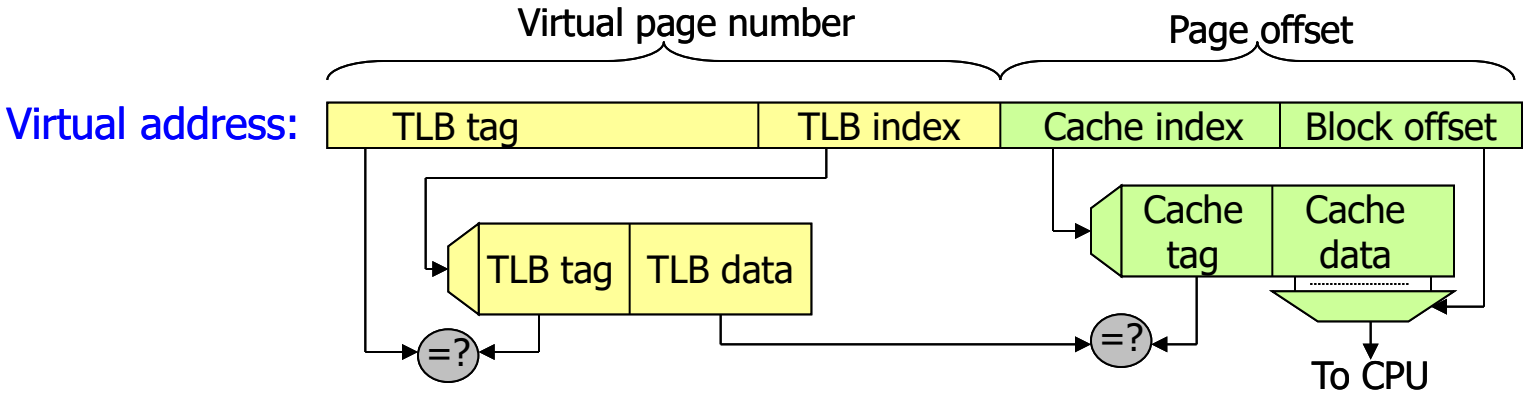
Integrating virtual memory, caches and TLB

Is there a way to achieve both, addressing the cache physically *and* saving the time for accessing the TLB?

Remember:

1. The address index selects a cache block, whose tag is compared with the address tag afterwards.
   ⇒ The address tag may arrive significantly later than the index; we can afford to translate it if we translate the address tag and select the cache tag in parallel!
2. The page offset is the same for virtual and physical address.
   ⇒ If the index is selected from the page offset no translation is necessary!

Virtually indexed and physically tagged Cache



Process Switches and TLB

Multi-Tasking Systems

Consider a switch from process P1 to process P2:

The page table register gets overwritten,

But: the entries in the TLB still point to physical pages belonging to process P1!

⇒ We have to prevent P2 from accessing pages of P1

Solutions:

1. Clear TLB at process switch
   - \+ Easy to implement
   - \- Inefficient if TLB is large enough to hold entries for all active processes
2. Extend the TLB tags by a process identifier

   Process identifier is stored in a CPU register and set by operating system when doing a context switch

Dealing with large page tables

How large are page tables?

Example:

Consider a 64-bit processor with a page size of 2^{16} = 64 KB

$\Rightarrow 2^{64}/2^{16} = 2^{48}$ pages!

Assume 8 byte per entry in page table:

$\Rightarrow$ 2048 TB (Terabyte!) for one page table

$\Rightarrow$ We cannot use one page table of full size for each process!

Basic Idea of all solutions:

Try to keep the page table small by providing entries only for those virtual pages where a physical page already has been allocated for by the operating system!

Multi tasking systems

Physical pages being shared by several processes:

- Sometimes pages containing shared objects may be read by several programs but may be modified only by one single process!
- Pages containing shared library routines may only be read but not be modified by programs running in user mode!

For each page table entry one Write access bit :

Indicates whether page may be written by the process owning the page table.

Multi tasking systems

Only the operating system is allowed to modify page tables to protect programs against each other.

But how does the CPU know whether the procedure being run belongs to the operating system?

CPU offers two operation modes:

1. Kernel mode: unrestricted
   Access to all areas of memory is granted
   All instructions offered may be executed

2. User mode: restricted
   Some instructions may not be executed, e.g. instruction for modifying the page table register and the TLB

Present CPU mode is indicated by a kernel/user status bit. If a user program wants to call a kernel mode procedure a system call exception is used.